



中国科学院大学
University of Chinese Academy of Sciences

Deep Learning

Regularization and optimization

Xinfeng Zhang (张新峰)

School of Computer Science and Technology

University of Chinese Academy of Sciences

Email: xfzhang@ucas.ac.cn



计算机科学与技术学院

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY



提纲

- 深度学习模型的训练与测试
- 参数范数正则化
- 数据增强
- Bagging
- 提前终止
- Dropout和DropConnect
- 归一化与优化



5

Dropout和DropConnect

Dropout的提出

❑ Dropout由Hinton在2012年提出。它计算简单、功能强大，大多数神经网络模型都可以使用

- Hinton在其论文《Improving neural networks by preventing co-adaptation of feature detectors》中提出Dropout (谷歌学术引用12,134次)
- Alex、Hinton在其论文《ImageNet Classification with Deep Convolutional Neural Networks》中用到了Dropout算法(这篇论文提出的AlexNet网络模型赢得了2012年图像识别大赛冠军, 谷歌学术引用142,984次)

❑ 目前为止，大多数的深度神经网络都使用了Dropout正则化方法

- [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)
- [Improving Neural Networks with Dropout](#)
- [Dropout as data augmentation](#)

[1] Hinton, Geoffrey E., Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R. Salakhutdinov. "Improving neural networks by preventing co-adaptation of feature detectors." arXiv preprint arXiv:1207.0580 (2012).

[2] Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." In Advances in neural information processing systems, pp. 1097-1105. 2012.

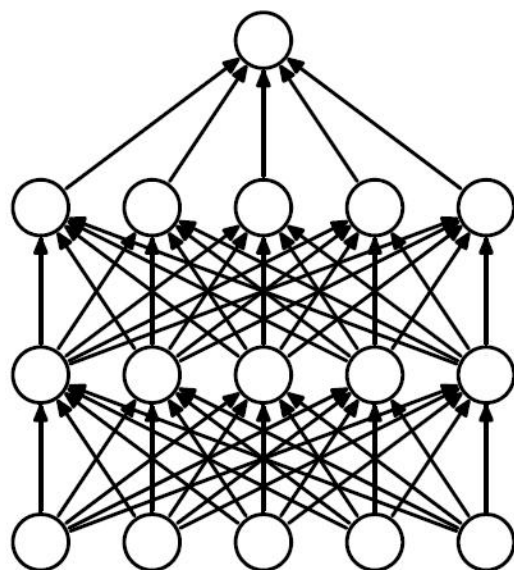
Dropout的工作过程

- ❑ 训练深度神经网络的一种技巧（Trick），在**每个训练批次中，删掉部分的神经元节点**，即在**网络前向传播的时候**，让**某些神经元的激活值以一定的概率 p 输出为0**
 - 输入层和隐藏层都可以dropout一些神经元
 - 隐藏层概率通常取0.5
 - 输入层dropout概率通常取0.2
- ❑ 这种方式**可以减少层间节点的相互作用**，因此可以明显地缓和过拟合现象
- ❑ 使模型泛化性更强，因为它不会太依赖某些局部的特征

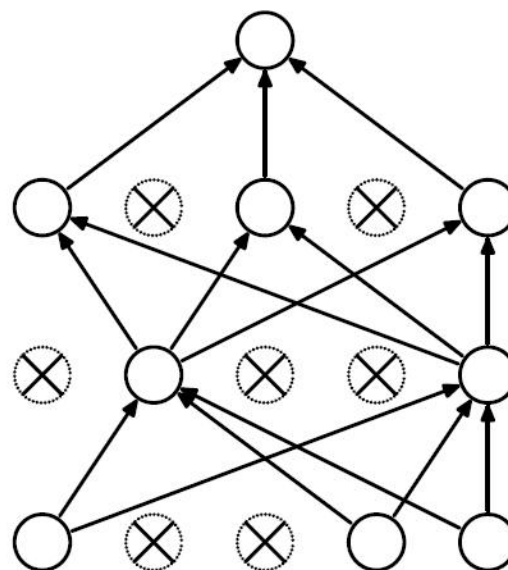
Dropout的工作过程

□ Dropout改变网络结构

- 相当于训练一个简化的网络”thinned network”



(a) Standard Neural Net



(b) After applying dropout.

Dropout的实现

□ 训练过程

- 将网络中的隐层神经元的激活值（输出）以概率 p 随机置0（每次迭代激活值置零的神经元不同）
- 对网络进行正向、反向传播，即训练网络，反向传播时激活值被置0神经元的参数保持没有被删除前的值，其他神经元的参数被更新
- 重复这一过程，直至训练结束

Dropout的实现

□ 训练过程

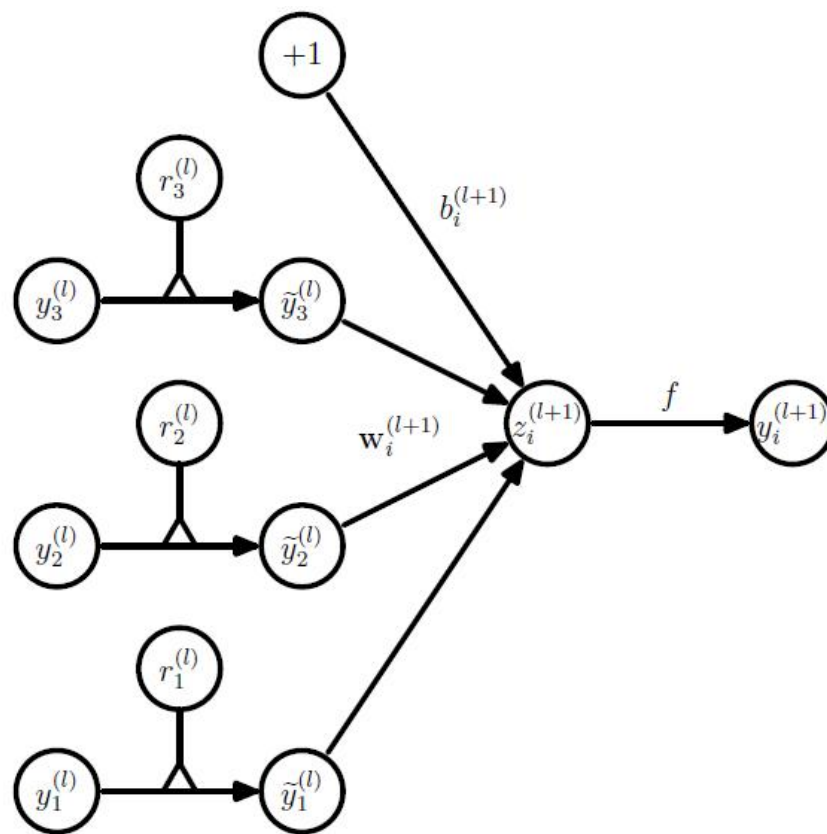
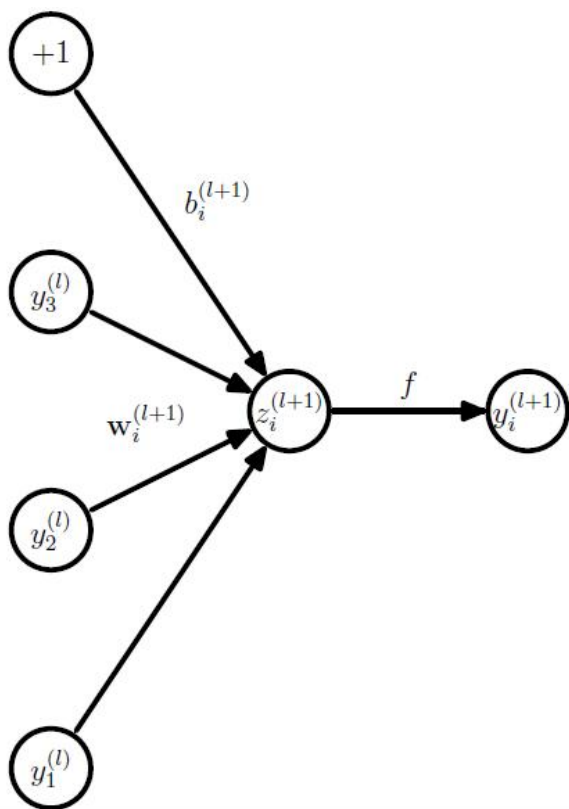
- 标准的网络结构和Dropout网络训练过程中前向计算

$$\begin{aligned} z_i^{(l+1)} &= \mathbf{w}_i^{(l+1)} \mathbf{y}^l + b_i^{(l+1)}, & r_j^{(l)} &\sim \text{Bernoulli}(p), \\ y_i^{(l+1)} &= f(z_i^{(l+1)}), & \tilde{\mathbf{y}}^{(l)} &= \mathbf{r}^{(l)} * \mathbf{y}^{(l)}, \\ & & z_i^{(l+1)} &= \mathbf{w}_i^{(l+1)} \tilde{\mathbf{y}}^l + b_i^{(l+1)}, \\ & & y_i^{(l+1)} &= f(z_i^{(l+1)}). \end{aligned}$$

Dropout的实现

□ 训练过程

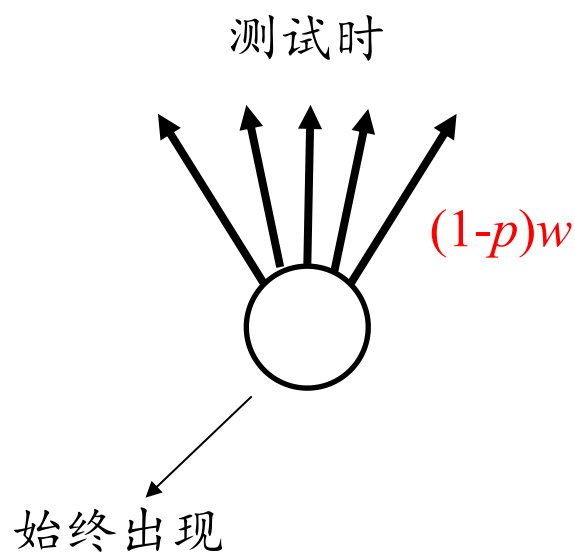
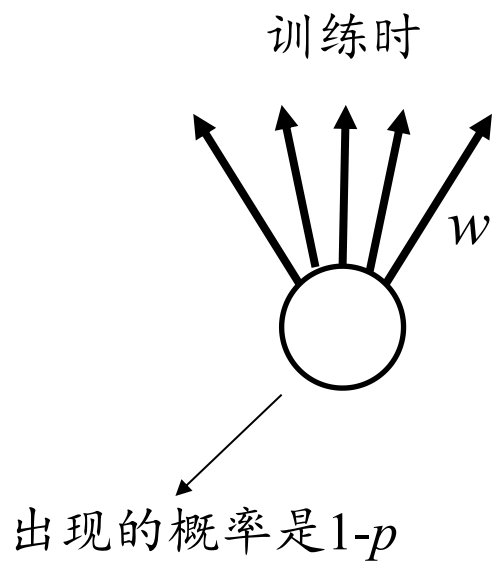
- 标准的网络结构和Dropout网络结构计算过程对比



Dropout的实现

□ 测试阶段

- 测试时使用所有神经元;
- 所有神经元的激活值需要乘以 $1-p$;

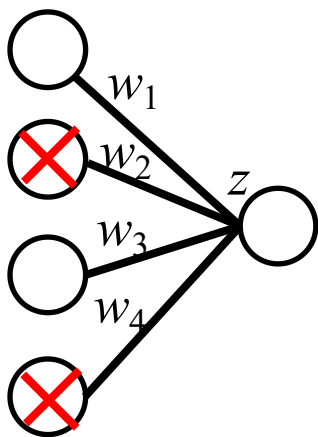


Dropout的实现

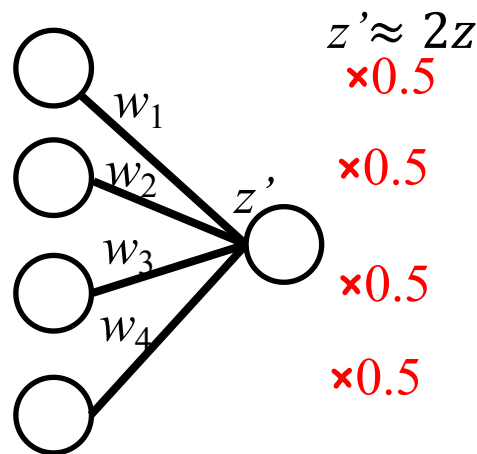
□ 测试阶段

- 测试时使用所有神经元；
- 所有神经元的激活值需要乘以 $1-p$;

训练时, 假设dropout rate 50%



测试时



Dropout的解释

□ 有效性的解释

- 不同的解释角度：数据增广，稀疏约束...
- 集成学习(Ensemble learning)

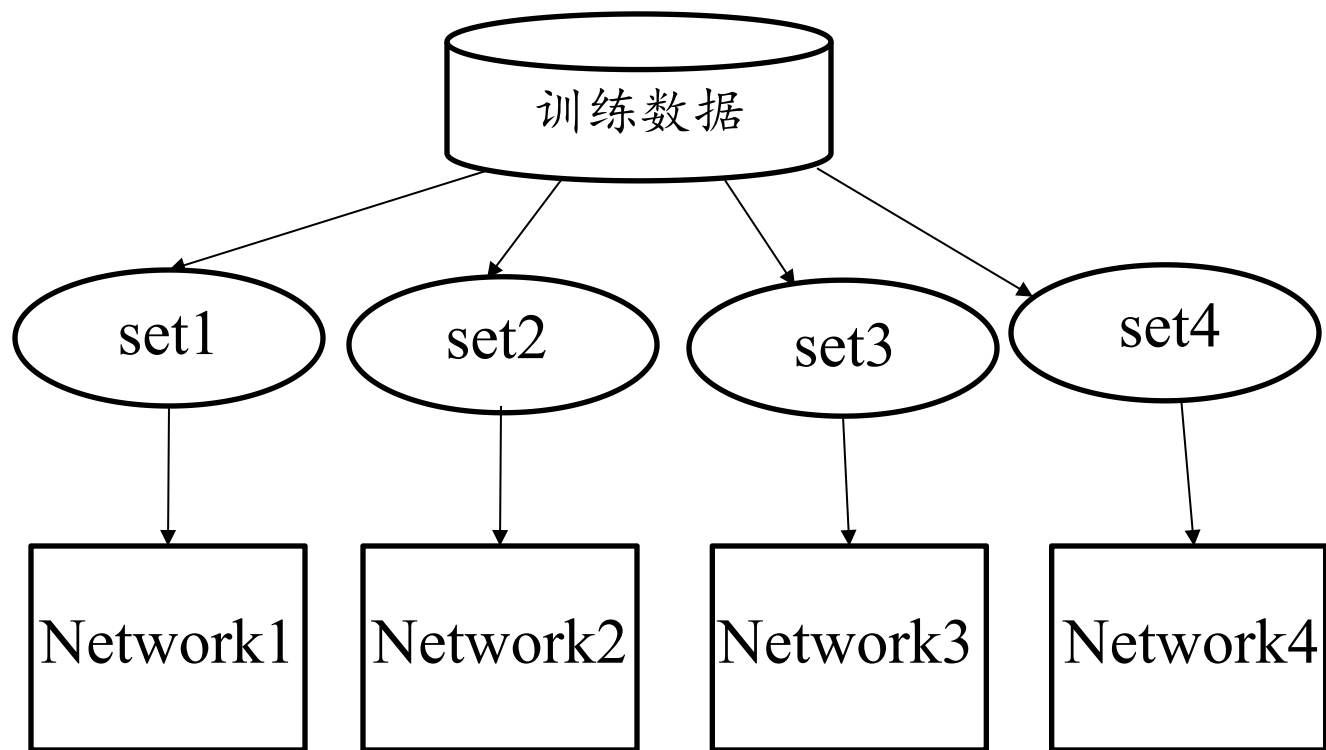
□ Dropout可以视作一种高效的近似Bagging的集成方法，并且是共享隐层单元的集成方法。可以理解为每次训练随机丢弃节点值，使得模型前后存在差异

Dropout的解释

- ❑ Dropout使得隐层节点不一定每次都出现，减少了隐层节点之间的复杂的共适应关系
- ❑ 这样权值的更新不再依赖于有固定关系的隐层节点的共同作用，阻止了某些特征仅仅在特定特征下才有效果的情况，迫使网络去学习更加鲁棒的特征

Dropout的解释

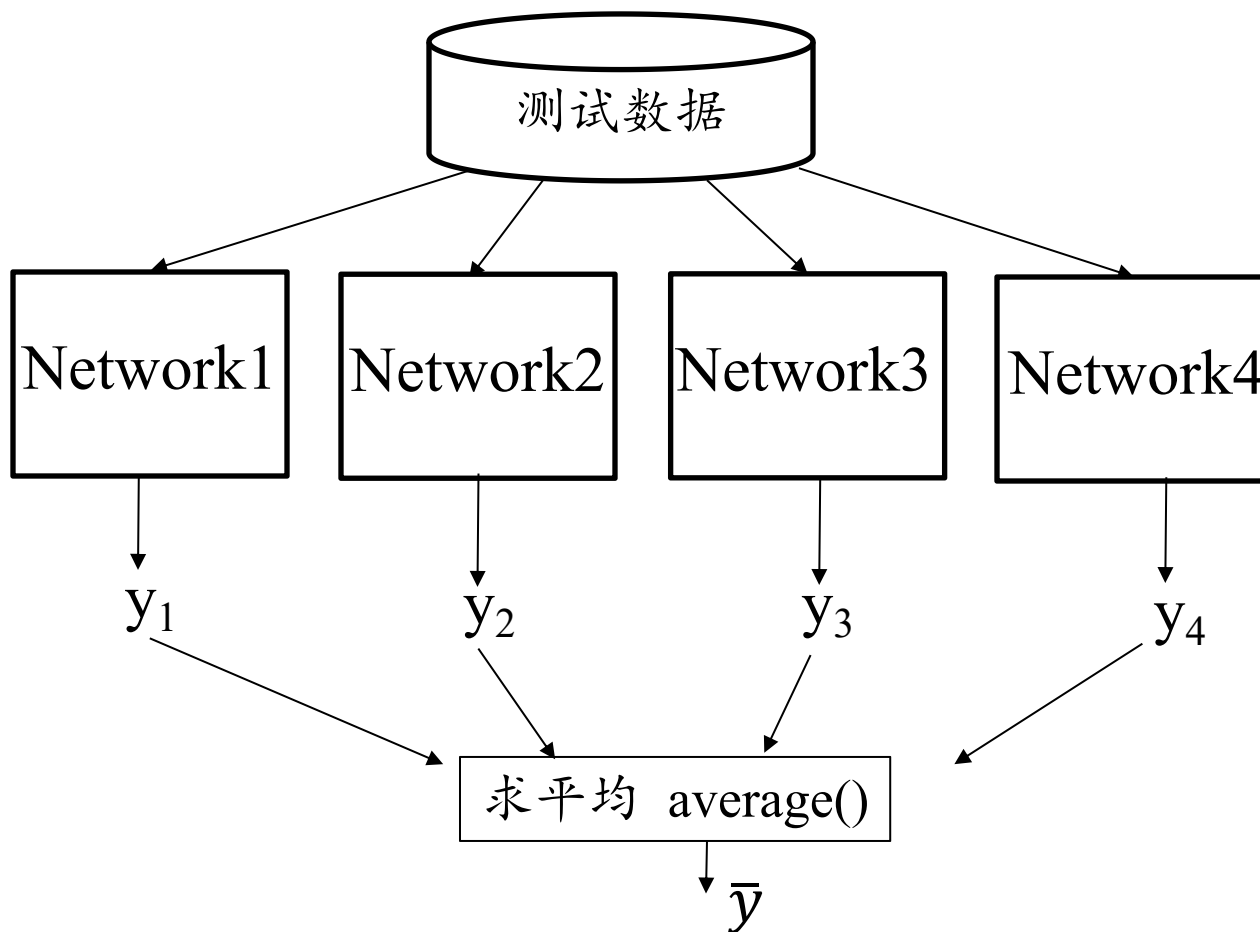
□ Dropout的近似Bagging集成方法解释



训练多个不同的结构的网络

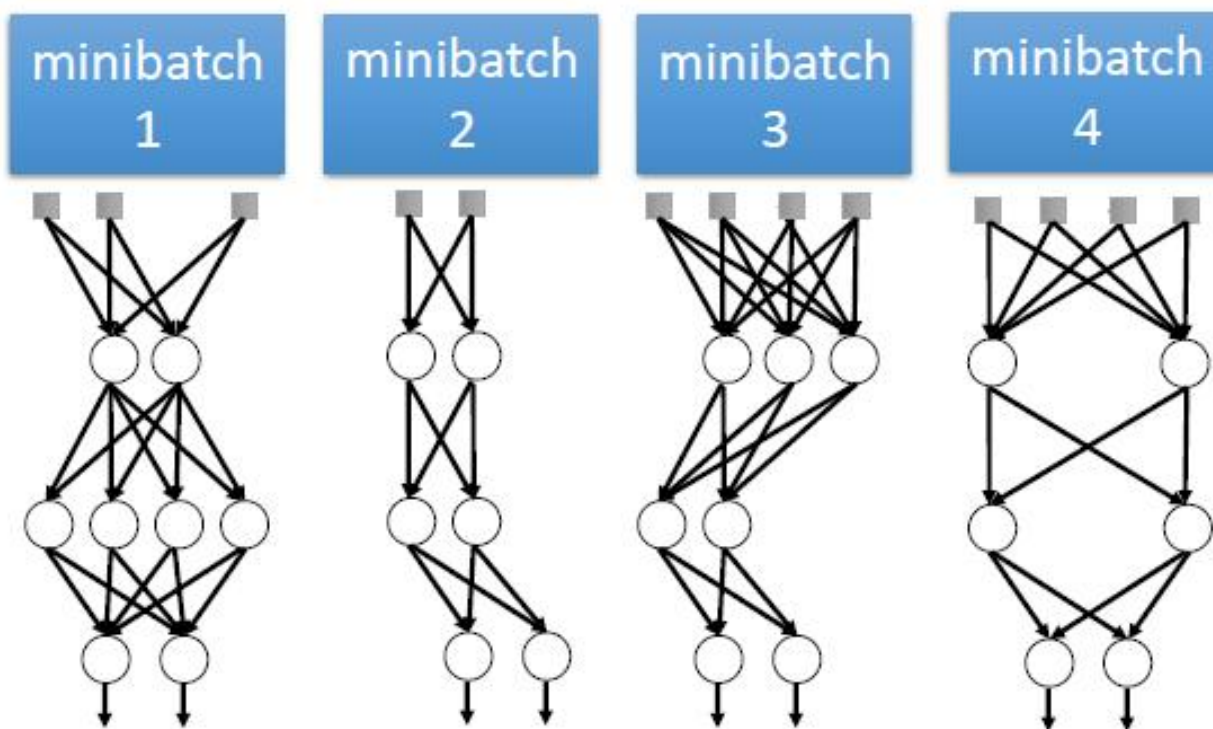
Dropout的解释

□ Dropout的近似Bagging集成方法解释



Dropout的解释

□ Dropout的近似Bagging集成方法解释

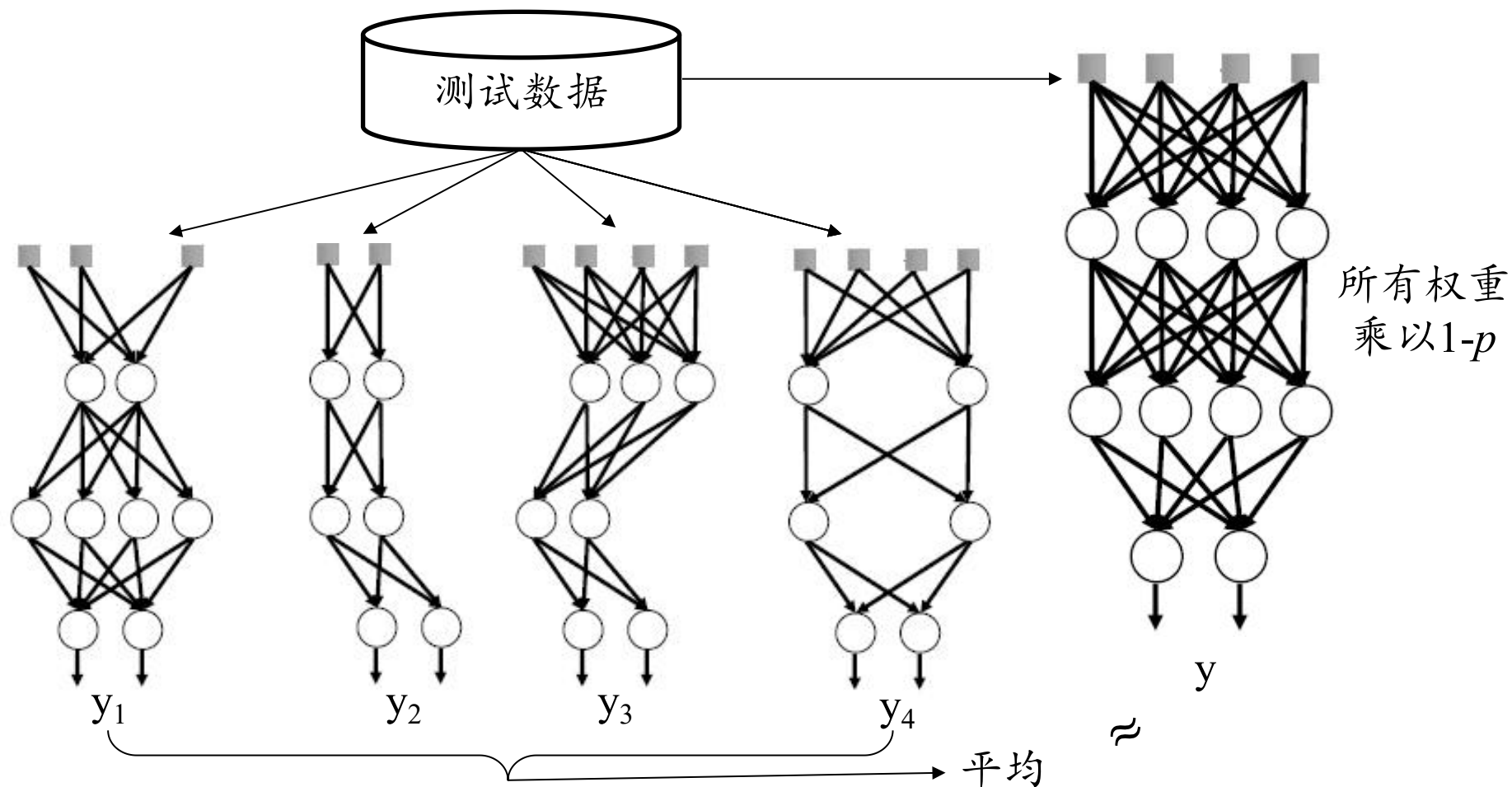


M 个神经元对应
有 2^M 可能的子网
络

- 每个minibatch训练一个子网络
- 不同子网络间神经元参数共享

Dropout的解释

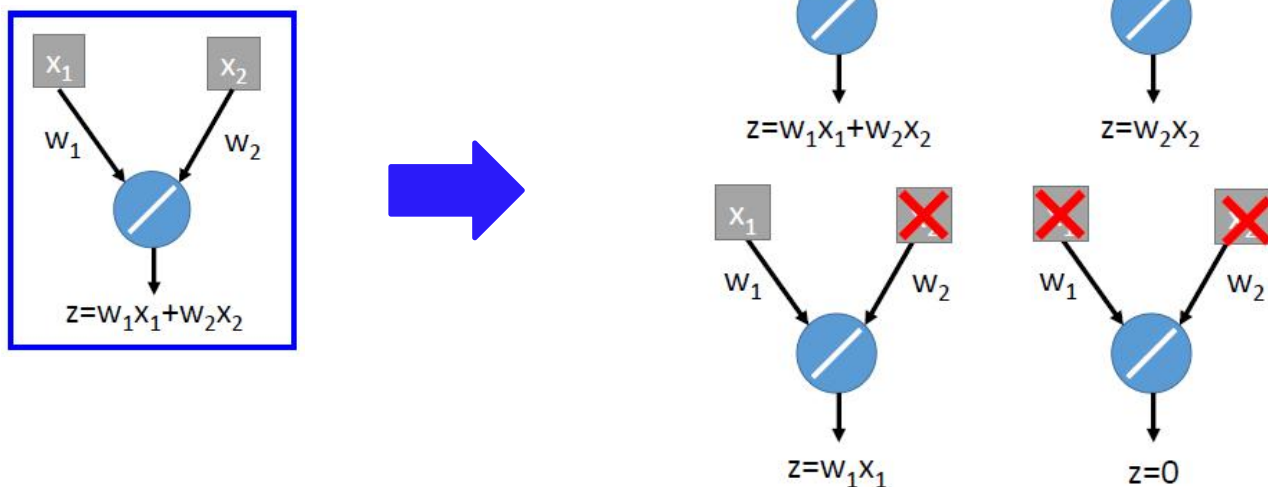
□ Dropout的近似Bagging集成方法解释



Dropout的解释

□ Dropout的近似Bagging集成方法解释

- 在神经网络接近linear时dropout性能会好，比如ReLU，Maxout激活函数



DropConnect

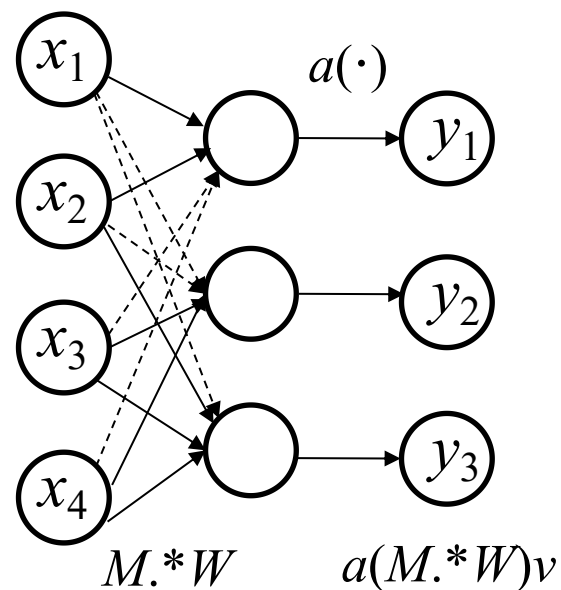
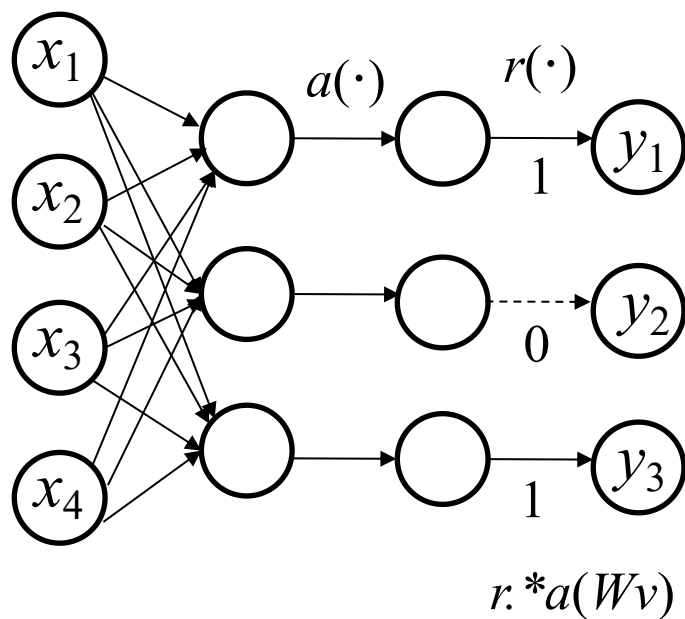
□ DropConnect也是通过舍弃一部分单元来防止过拟合问题的方法

- DropConnect可以认为是Dropout的扩展。它是丢掉神经元和神经元之间的连接，包含权重和偏置，即不需要进行激活操作前的线性变换
- 它比Dropout效率更高且性能得到提升，但两者还是比较相似，差别不大

Wan, Li, Matthew Zeiler, Sixin Zhang, Yann Le Cun, and Rob Fergus. "Regularization of neural networks using dropconnect." In International conference on machine learning, pp. 1058-1066. 2013.

DropConnect

□ DropConnect与Dropout的异同



DropConnect

□ 使用DropConnect和Dropout在不同数据集上的测试误差

- 虽然DropConnect的识别性能优于Dropout，但是其训练难度也高于Dropout

数据集	Dropout	DropConnect
MNIST	0.27	0.21
CIFAR-10	9.83	9.32
SVHN	1.96	1.94
NORB	3.03	3.23



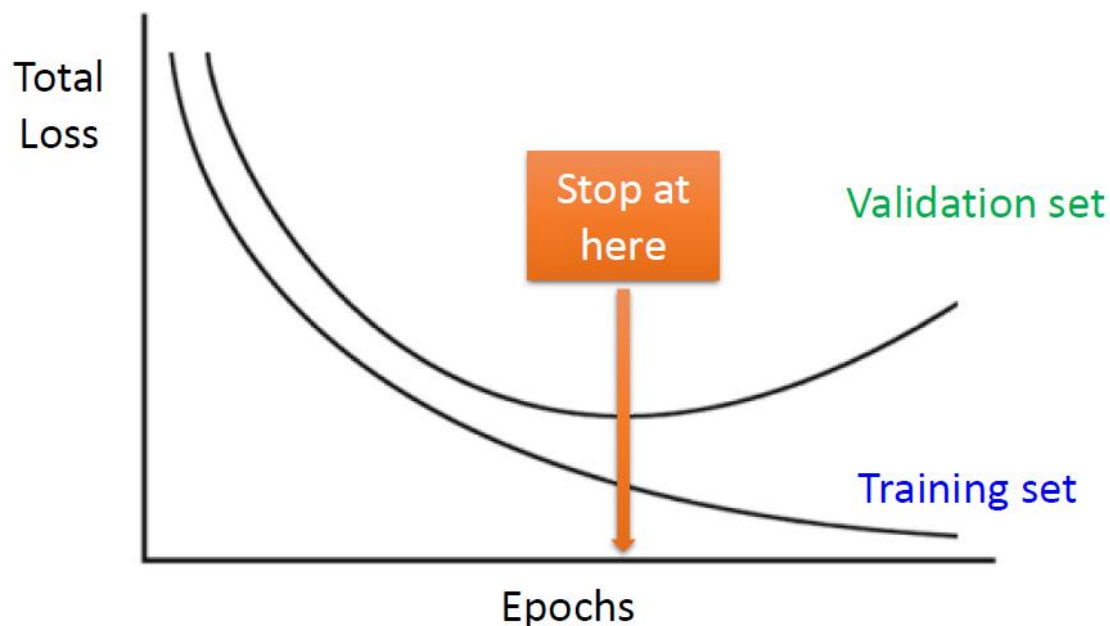
6

提前终止

提前终止

□ 随着训练迭代次数的增加，训练误差也在逐步降低。但是会发现验证集损失先下降后在某个节点开始上升的现象

- 提前终止（Early stopping）目的则在于找到这个上升的时间点，并及时终止模型的训练
- 提前终止是深度学习中最常用的正则化方法，可单独使用或与其他正则化策略结合使用



提前终止实现

□ 需要尽量避免训练波动导致的提前终止

提前终止算法

令 n 为评估间隔的步数，在验证集上连续出现 p 次较坏结果后终止，设 θ_0 为初始化的

参数

$\theta \leftarrow \theta_0, \theta^* \leftarrow \theta$

$i \leftarrow 0, j \leftarrow 0$

$v \leftarrow \infty$

$i^* \leftarrow i$

while $j < p$ do

 执行 n 步训练，同时对 θ 进行更新

$i \leftarrow i + n$

$v' \leftarrow \text{ValidationSetError}(\theta)$

 if $v' < v$ then

$j \leftarrow 0$

$\theta^* \leftarrow \theta$

$i^* \leftarrow i$

$v \leftarrow v'$

 else

$j \leftarrow j + 1$

 end if

end while

得到最佳参数 θ^* ，最佳训练步数 i^*

提前终止算法：额外训练

- 使用提前终止策略过程中需要使用到验证集（Validation Set），这也就意味着在训练过程中会有一部分数据无法进行训练。
- 数据宝贵，为了更好地利用验证集中的数据，我们可以在完成提前终止后将验证集中的数据加入到训练集中，进行额外的训练

提前终止算法：额外训练

□ 策略一

- 在进行额外训练前，已经通过提前终止确定了训练的**最佳步数**。使用**所有的数据**进行训练时，在确定的**最佳步数**时终止训练。
- 但是值得指出的是，我们**不能确定按照提前终止确定的训练最佳步数再次训练时仍能得到一个最佳的训练**，因为在进行额外训练时训练集变大了，因此在训练过程中对参数的更新次数也会变多

策略 1 利用提前终止确定的最佳训练步数在所有数据上再次训练

已有 $(X^{all_train}, y^{all_train})$ 为全部训练集

将 $(X^{all_train}, y^{all_train})$ 划分为 (X^{train}, y^{train}) 和 (X^{val}, y^{val})

随机初始化参数 θ ，使用 (X^{train}, y^{train}) 作为训练集， (X^{val}, y^{val}) 作为验证集进行训练，利用提前终止返回最佳训练步数 t

再次随机初始化参数 θ ，使用 $(X^{all_train}, y^{all_train})$ 作为训练集上训练 t 步

提前终止算法：额外训练

□ 策略二

- 在提前终止时得到的模型的基础上再次利用所有的训练集进行训练，在这种情况下我们已经没有验证集可以供我们判断何时终止训练，但是我们可以通过监控验证集的平均损失函数，直到它低于之前提前终止时在训练集上的目标损失值
- 这个策略相较于上一个策略避免了重新训练模型，但是这种策略甚至无法保证继续训练是否能达到之前的目标值

策略 2 利用提前终止确定将会过拟合的目标损失值，然后在所有数据上再次训练直到再次到达该值

已有 $(X^{all_train}, y^{all_train})$ 为全部训练集

将 $(X^{all_train}, y^{all_train})$ 划分为 (X^{train}, y^{train}) 和 (X^{val}, y^{val})

随机初始化参数 θ ，使用 (X^{train}, y^{train}) 作为训练集， (X^{val}, y^{val}) 作为验证集进行训练，利用提前终止确定目标损失值 ϵ

$\epsilon \leftarrow J(\theta, X^{train}, y^{train})$

在 $(X^{all_train}, y^{all_train})$ 继续训练，直到 $J(\theta, X^{val}, y^{val}) \leq \epsilon$



7

归一化与优化



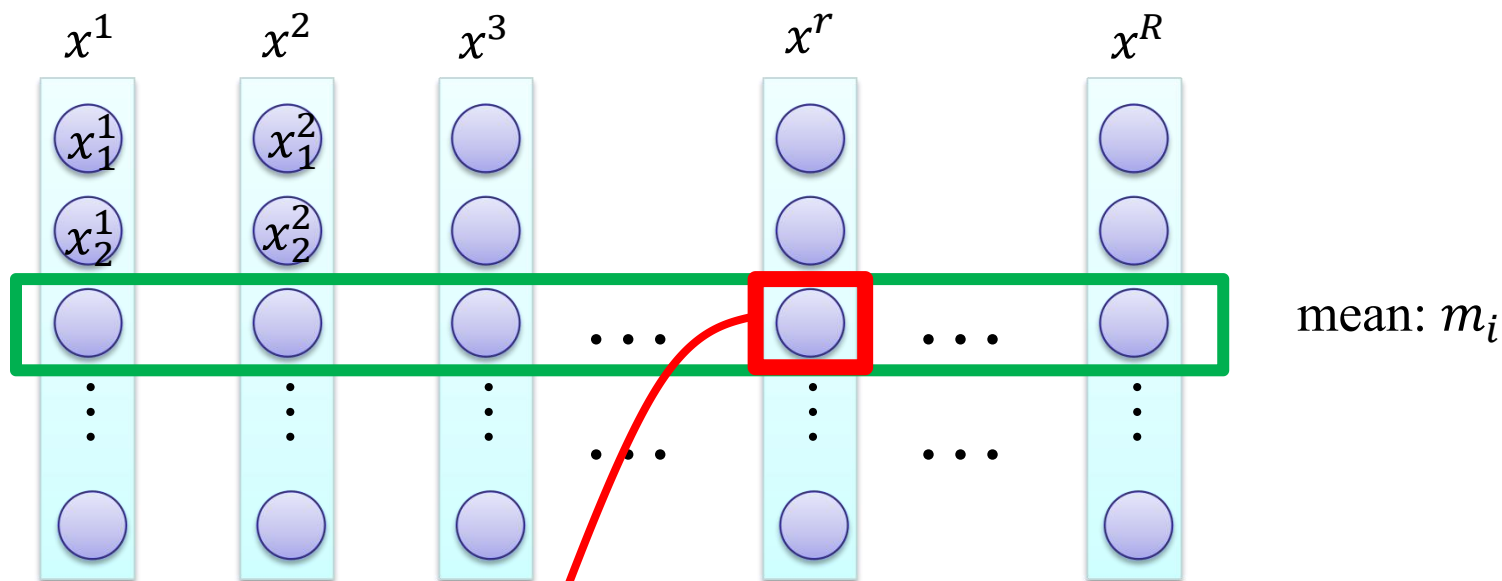
归一化

- ❑ 由于样本不同特征的量纲或量纲单位不同，变化区间也处于不同数量级，使用原始样本训练机器学习或深度学习模型，不同特征对模型的影响会不同，这也导致训练所得的模型缺乏鲁棒性。
- ❑ 归一化（Normalization）是一种常见的数据预处理方法，它将数据映射成 $[0, 1]$ 或者 $[-1, 1]$ 之间的小数，是的模型更容易收敛到最优解。
- ❑ 因此，在构造机器学习或深度学习模型之前，需要将数据做归一化处理，这不仅会加快求解的速度，还有可能提高模型的精度。



数据预处理

□ Feature scaling



$$x_i^r \leftarrow \frac{x_i^r - m_i}{\sigma_i}$$

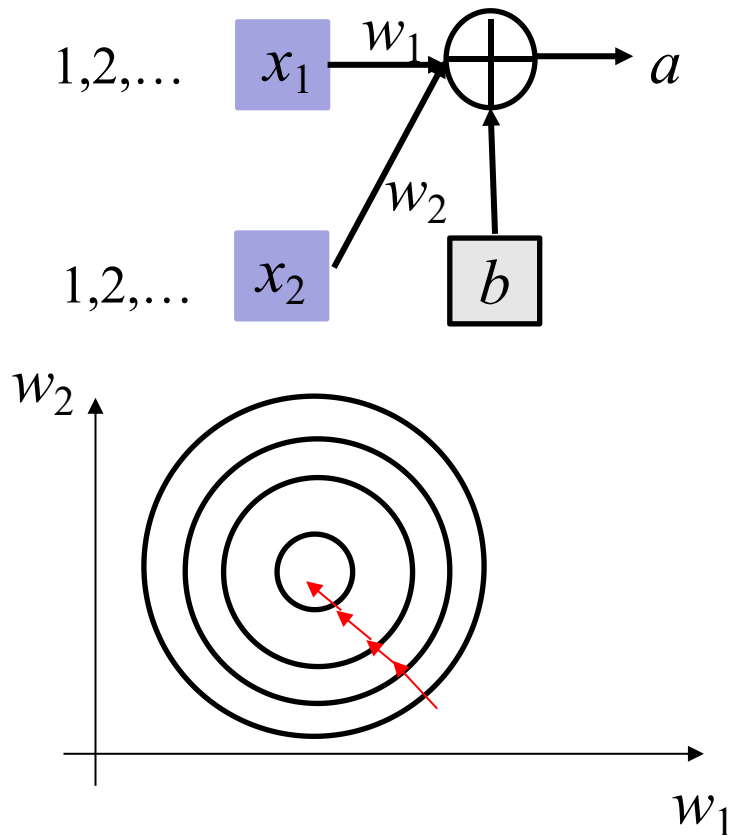
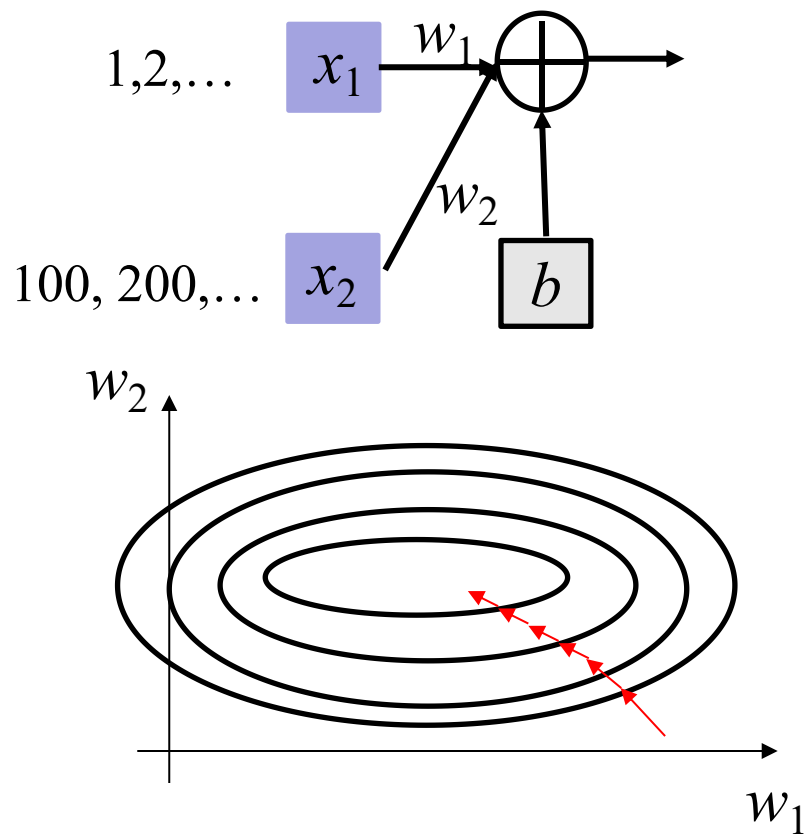
处理后所有维度均值为0方差为1

通常处理后会使得训练收敛加快



数据预处理

□ Feature scaling



最大最小归一化

- 给定样本集合 $X=\{x_i\}$, $i=1,\dots,n$, $x^{\min}$ 为样本的最小值, $x^{\max}$ 为样本的最大值。
- 最大最小归一化 (Max-min Normalization) 将数据归一化到 $[0, 1]$ 之间, 计算公式如下:

$$x_i^{\min\max} = \frac{x_i - x^{\min}}{x^{\max} - x^{\min}}$$



最大最小归一化

- ❑ 最大最小归一化可以很好地将数据归一化到 $[0, 1]$ 之间，使数据具有相同的度量尺度。但是这种归一化方法容易受到离群点数据的影响，如果存在一个离群点数据的值过大或者过小，都会导致分母很大，这使得归一化后的数据失去意义。
- ❑ 另外，最大最小归一化后的数据值都为正数，这导致机器学习模型的参数只能同时增大或同时减小，参数的更新路线可能不是最优路线。



标准归一化

- 标准归一化 (Standard Normalization) 考虑所有样本数据, 因此受离群点数据的影响要小一些, 尤其在样本足够多的情况下表现比较稳定。
- 给定样本集合 $X=\{x_i\}, i=1,\dots,n$, $E(X)$ 为样本均值, $Var(X)$ 为样本方差, 标准归一化的计算公式如下:

– 其中:

$$x_i^{stand} = \frac{x_i - E(X)}{\sqrt{Var(X)}}$$

$$E(X) = \frac{1}{n} \sum_{i=1}^n x_i$$

$$Var(X) = \frac{1}{n} \sum_{i=1}^n (x_i - E(X))^2$$



标准归一化

- 标准归一化的分子得到的值有正有负，可以让不同的参数朝着不同方向调整直至到达最优解，减少迭代次数，使得更新尽可能沿着最优路径进行。
- 标准归一化的分母用到了方差，方差衡量的是数据的离散程度，因此可以减小离群点数据的影响。



批归一化

- ❑ 在神经网络装载数据时，通常将按批（Batch）输入，由于每批数据的分布并不相同，使模型训练起来比较困难。
- ❑ 并且在训练深层神经网络时，激活函数会改变各层数据的分布和量级，随着神经网络的加深，这种改变会越来越大，从而导致模型不够稳定、不容易收敛，甚至可能出现梯度消失的问题。
- ❑ 这些存在的问题可以通过批归一化（Batch Normalization, BN）来解决。



批归一化

- 神经网络中传递的张量数据一般有四个维度，记为 (N, H, W, C) ，其中 N 表示批大小， W 、 H 分别表示特征图的宽和高， C 表示通道数量， $N, H, W, C \in \mathbb{N}$ 。
- 批归一化是让一个批次内的所有数据在某个通道维度上的均值为0，方差为1，即在 N 、 H 、 W 维度上做归一化。这样不仅可以使数据分布一致，而且可以避免梯度消失的发生。



批归一化

□ 给定一个批大小为N的样本集合 $X \in \mathbb{R}^{N \times H \times W \times C}$ 作为输入

- $E_c[X]$ 为样本沿着通道方向计算得到的均值
- $Var_c[X]$ 为样本沿着通道方向计算得到的方差
- ϵ 是一个很小的数，防止分母为零
- $\gamma \in \mathbb{R}^C$ 为可学习放缩 (Scale) 向量，向量元素的初始值通常设置为1， $\beta \in \mathbb{R}^C$ 为可学习平移 (Shift) 向量，向量元素的初始值通常设置为0。
- 批归一化的计算公式：

$$E_c[X] = \frac{1}{N \times H \times W} \sum_{n=1}^N \sum_{h=1}^H \sum_{w=1}^W \underline{X}_{n,h,w,\cdot}$$

$$Var_c[X] = \frac{1}{N \times H \times W} \sum_{n=1}^N \sum_{h=1}^H \sum_{w=1}^W (X_{n,h,w,\cdot} - E_c[X])^2$$

$$Y_{n,h,w,\cdot} = \frac{X_{n,h,w,\cdot} - E_c[X]}{\sqrt{Var_c[X] + \epsilon}} \times \gamma + \beta$$



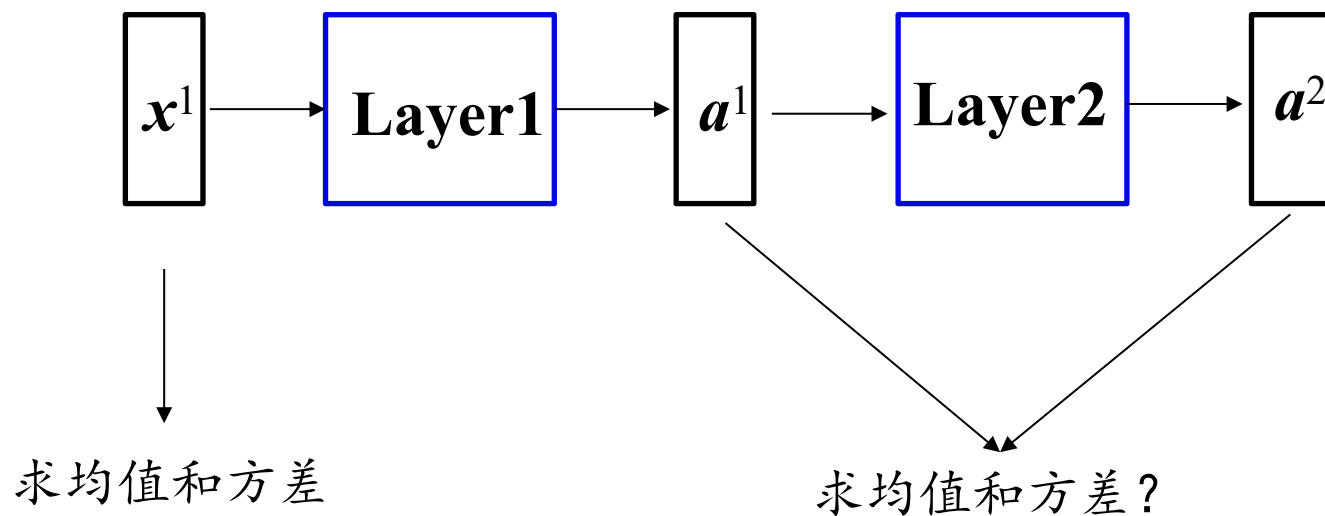
批归一化

- ❑ 批归一化使网络中每层输入数据的分布相对稳定，减少梯度消失和梯度爆炸问题，提高模型的稳定性，也使得神经网络更快地收敛到最优解。
- ❑ 同时，它使模型对输入数据中的一些变化和扰动具有更好的鲁棒性，提高了模型的泛化能力。
- ❑ 但是批归一化受到批大小影响较大。如果批大小过小，计算的均值和方差不足以代表整个样本数据的分布，批归一化的效果可能会下降；如果批大小过大，会超过内存容量，无法处理。
- ❑ 此外，批归一化并不适合所有神经网络，如循环神经网络等动态网络，使用批归一化的效果并不好。



Batch Normalization

□ 神经网络中的feature scaling

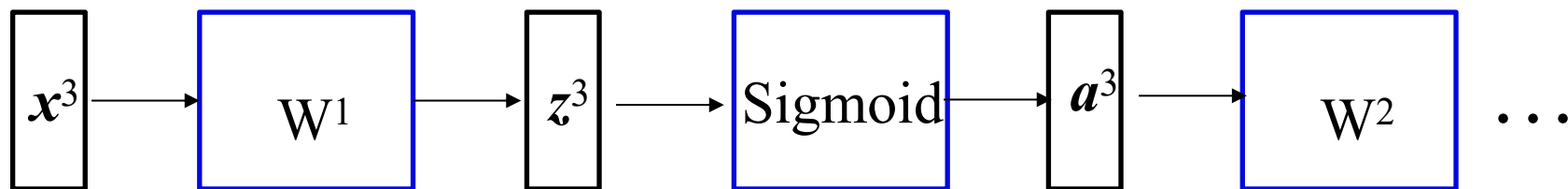
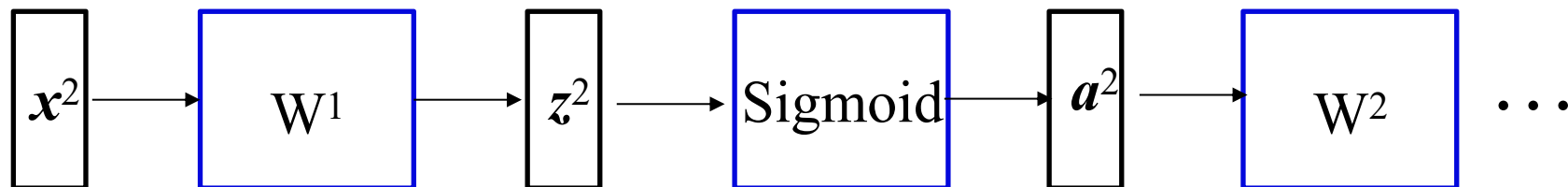
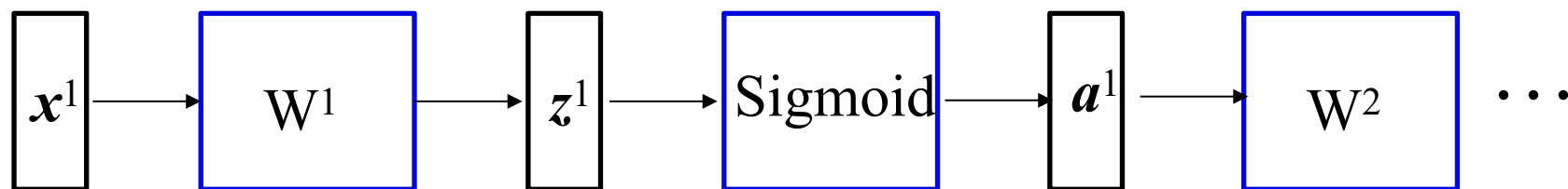


Ioffe, Sergey, and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift." In International conference on machine learning, pp. 448-456. PMLR, 2015.

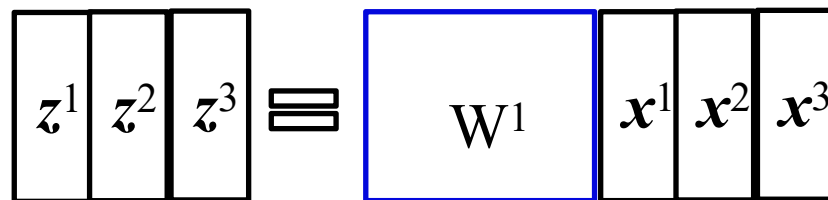


Batch Normalization

□ 神经网络中的feature scaling

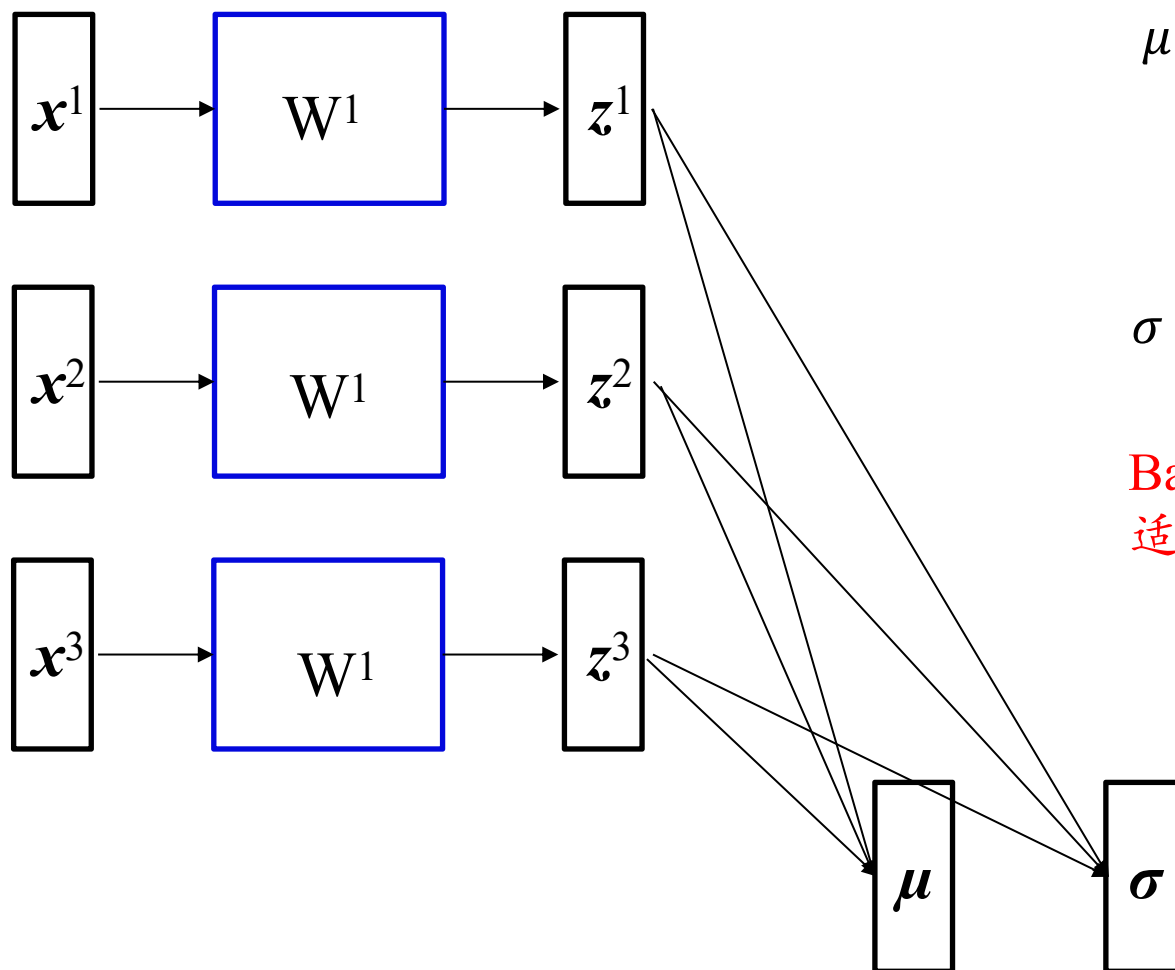


Batch



Batch Normalization

□ 神经网络中的feature scaling



$$\mu = \frac{1}{N} \sum_{i=1}^N z^i$$

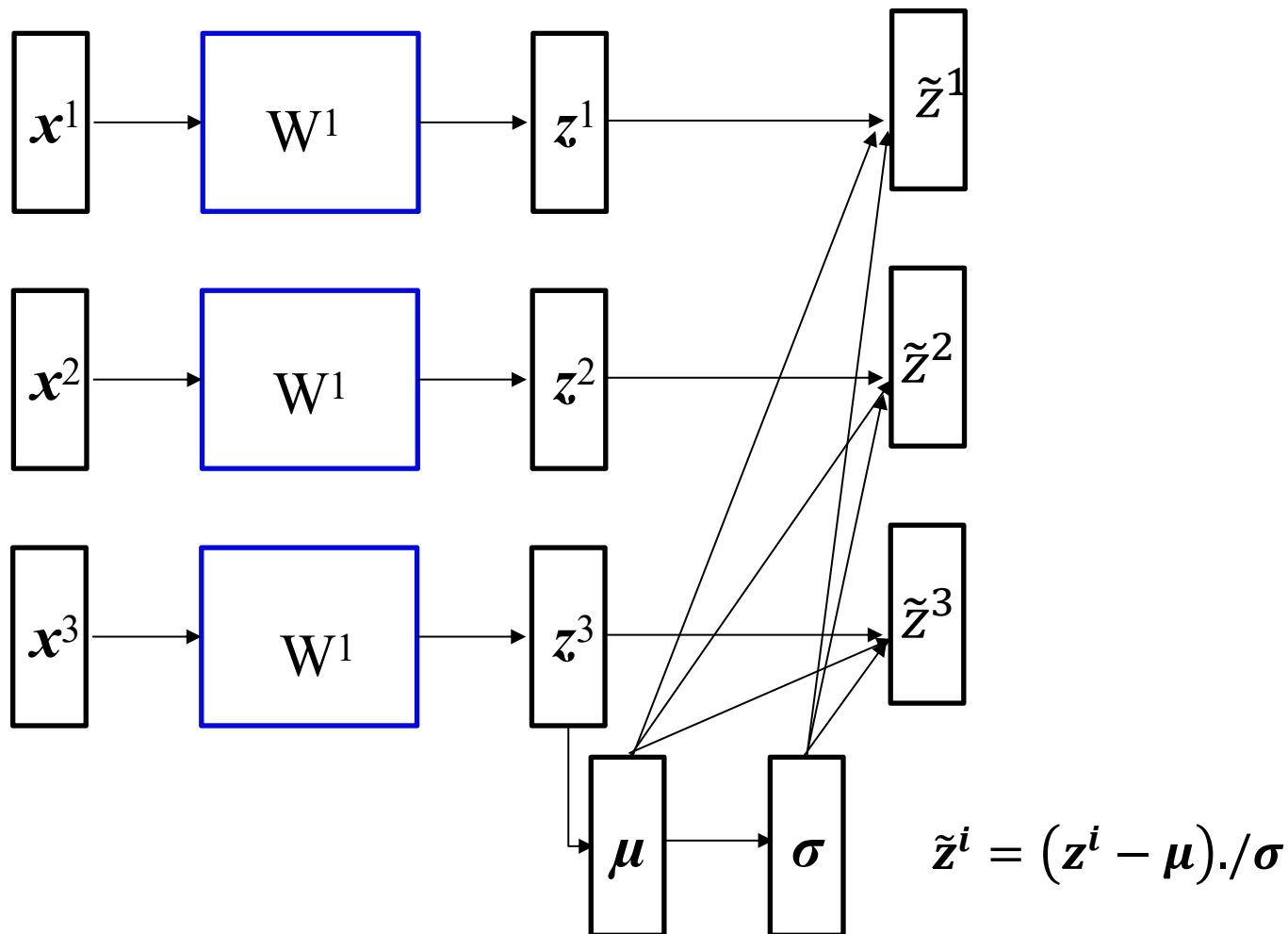
$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (z^i - \mu)^2}$$

Batch normalization不适用于较小的batch



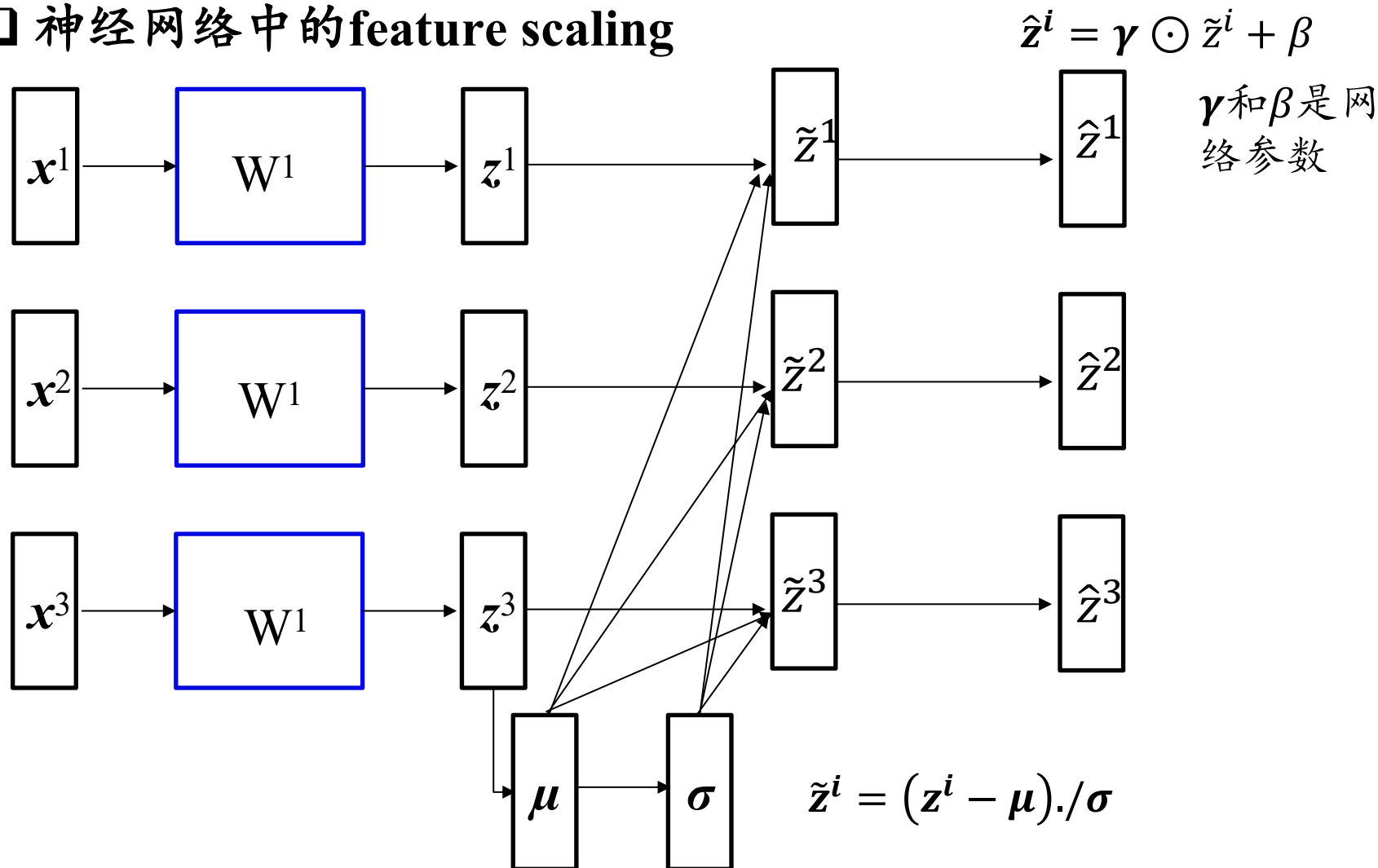
Batch Normalization

□ 神经网络中的feature scaling



Batch Normalization

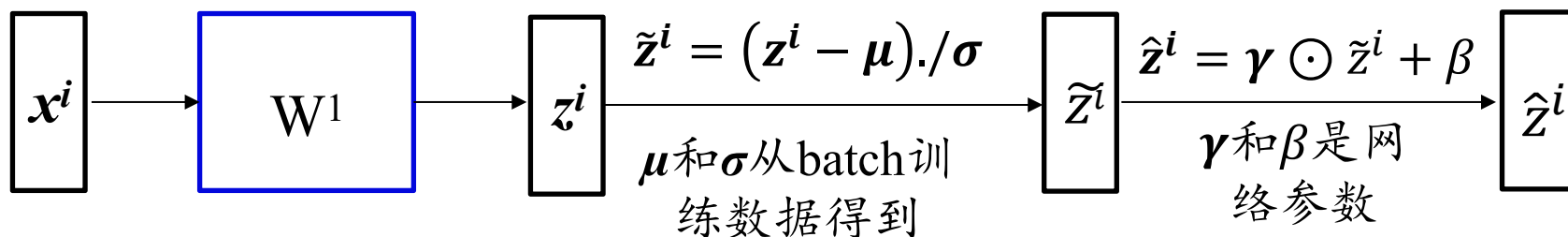
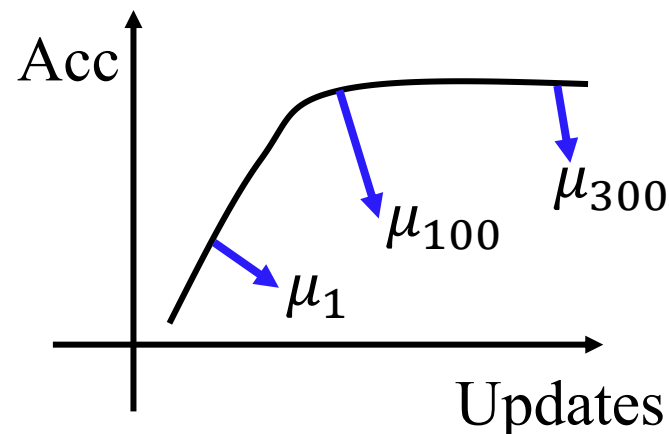
□ 神经网络中的feature scaling



Batch Normalization

□ 神经网络中的feature scaling

– 在测试阶段如何得到 μ 和 σ ?



Ideal solution:

Computing μ and σ using the whole training dataset.

Practical solution:

Computing the moving average of μ and σ of the batches during training.



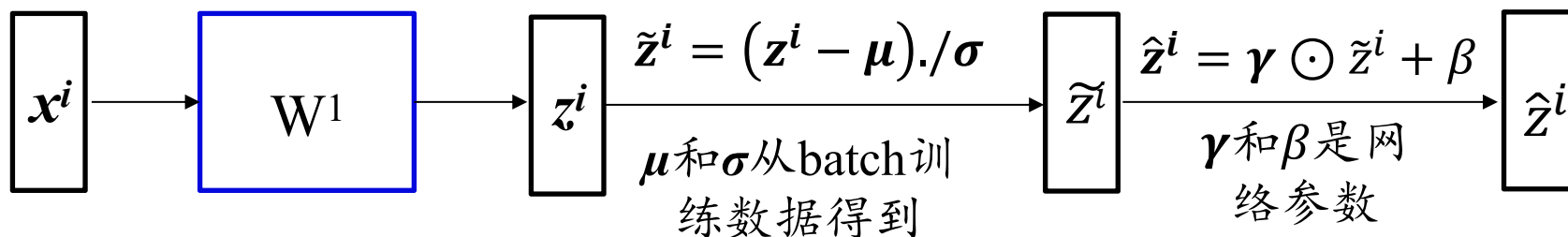
Batch Normalization

□ Batch Normalization的流程

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$
$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$
$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.



Batch Normalization

□ Batch Normalization性能分析

- Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

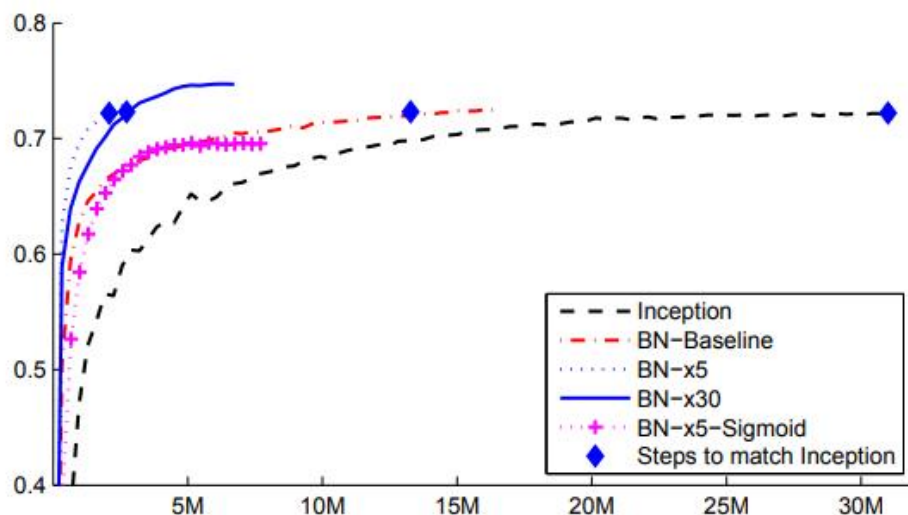


Figure 2: Single crop validation accuracy of Inception and its batch-normalized variants, vs. the number of training steps.

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
BN-Baseline	$13.3 \cdot 10^6$	72.7%
BN-x5	$2.1 \cdot 10^6$	73.0%
BN-x30	$2.7 \cdot 10^6$	74.8%
BN-x5-Sigmoid		69.8%

Figure 3: For Inception and the batch-normalized variants, the number of training steps required to reach the maximum accuracy of Inception (72.2%), and the maximum accuracy achieved by the network.



Batch Normalization

□ Batch Normalization性能分析

- [Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift](#)

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
BN-Baseline	$13.3 \cdot 10^6$	72.7%
BN-x5	$2.1 \cdot 10^6$	73.0%
BN-x30	$2.7 \cdot 10^6$	74.8%
BN-x5-Sigmoid		69.8%

Figure 3: *For Inception and the batch-normalized variants, the number of training steps required to reach the maximum accuracy of Inception (72.2%), and the maximum accuracy achieved by the network.*

Interestingly, increasing the learning rate further (BN-x30) causes the model to train somewhat slower initially, but allows it to reach a higher final accuracy. This phenomenon is counterintuitive and should be investigated further.



层归一化

- 在自然语言处理等任务中，不同样本的长度往往是不同的，而批归一化很难满足需求。为了解决这一问题，通常使用层归一化（Layer Normalization, LN）方法。
- 层归一化是指让一个数据样本在所有通道上的均值为0，方差为1，即在 H 、 W 、 C 维度上做归一化。



层归一化

□ 给定一个批大小为N的样本集合 $\mathbf{X} \in \mathbb{R}^{N \times H \times W \times C}$ 作为输入

- $E_n[\mathbf{X}]$ 是沿着批方向计算得到的均值
- $Var_n[\mathbf{X}]$ 是沿着批方向计算得到的方差
- ϵ 是一个很小的数，防止分母为零
- $\boldsymbol{\gamma} \in \mathbb{R}^C$ 为可学习放缩向量，向量元素的初始值通常设置为1
- $\boldsymbol{\beta} \in \mathbb{R}^C$ 为可学习平移向量，向量元素的初始值通常设置为0
- 层归一化的计算公式：

$$E_n[\mathbf{X}] = \frac{1}{H \times W \times C} \sum_{h=1}^H \sum_{w=1}^W \sum_{c=1}^C \mathbf{X}_{:,h,w,c}$$
$$Var_n[\mathbf{X}] = \frac{1}{H \times W \times C} \sum_{h=1}^H \sum_{w=1}^W \sum_{c=1}^C (\mathbf{X}_{:,h,w,c} - E_n[\mathbf{X}])^2$$
$$\mathbf{Y}_{:,h,w,c} = \frac{\mathbf{X}_{:,h,w,c} - E_n[\mathbf{X}]}{\sqrt{Var_n[\mathbf{X}] + \epsilon}}$$
$$\mathbf{Z}_{n,h,w,\cdot} = \mathbf{Y}_{n,h,w,\cdot} \times \boldsymbol{\gamma} + \boldsymbol{\beta}$$



层归一化

- ❑ 层归一化的最大优点是不依赖于批量数据的大小，甚至可以在单个样本上应用，另外它也不需要保存批的均值和方差，节省了存储空间。
- ❑ 此外，层归一化适用于循环神经网络。
- ❑ 但是，层归一化在批大小比较大时没有批归一化效果好。



实例归一化

- 在图像风格变换任务中，生成图像的风格主要依赖于某个图像实例，因而实例归一化（Instance Normalization）提出只对 H 、 W 维度进行归一化，即只让一个单独数据在 H 和 W 维度上求均值和方差。



实例归一化

□ 给定一个批大小为N的样本集合 $\mathbf{X} \in \mathbb{R}^{N \times H \times W \times C}$ 作为输入

- $E_{nc}[\mathbf{X}]$ 是沿着批方向和通道方向计算得到的均值
- $Var_{nc}[\mathbf{X}]$ 是沿着批方向和通道方向计算得到的方差
- ϵ 是一个很小的数，防止分母为零
- $\boldsymbol{\gamma} \in \mathbb{R}^C$ 为可学习放缩向量，向量的元素初始值通常设置为1
- $\boldsymbol{\beta} \in \mathbb{R}^C$ 为可学习平移向量，向量元素的初始值通常设置为0
- 实例归一化的计算公式：

$$E_{nc}[\mathbf{X}] = \frac{1}{H \times W} \sum_{h=1}^H \sum_{w=1}^W \mathbf{X}_{:,h,w,:}$$

$$Var_{nc}[\mathbf{X}] = \frac{1}{H \times W} \sum_{h=1}^H \sum_{w=1}^W (\mathbf{X}_{:,h,w,:} - E_{nc}[\mathbf{X}])^2$$

$$\mathbf{Y}_{:,h,w,:} = \frac{\mathbf{X}_{:,h,w,:} - E_{nc}[\mathbf{X}]}{\sqrt{Var_{nc}[\mathbf{X}] + \epsilon}}$$

$$\mathbf{Z}_{n,h,w,:} = \mathbf{Y}_{n,h,w,:} \times \boldsymbol{\gamma} + \boldsymbol{\beta}$$



组归一化

- ❑ 组归一化（Group Normalization）用来解决当批大小较小时批归一化效果较差的问题。层归一化虽然不依赖批大小，但是在卷积神经网络中直接对当前层所有通道数据进行归一化也并不理想。
- ❑ 组归一化将通道方向分组，然后在每个组内做归一化，使得一个数据样本在同一组内的所有通道的均值为0，方差为1。计算步骤与批归一化一样，但是不考虑批大小，即与批大小无关。



组归一化

□ 给定一个批大小为N的样本集合 $\mathbf{X} \in \mathbb{R}^{N \times H \times W \times C}$ 作为输入

- $E_{ng}[\mathbf{X}]$ 是沿着批方向在每个通道组内计算得到的均值
- $Var_{ng}[\mathbf{X}]$ 是沿着批方向在每个通道组内计算得到的方差
- ϵ 是一个很小的数，防止分母为零
- $\boldsymbol{\gamma} \in \mathbb{R}^C$ 为可学习放缩向量，向量元素的初始值通常设置为1
- $\boldsymbol{\beta} \in \mathbb{R}^C$ 为可学习平移向量，向量元素的初始值通常设置为0，其中G为分组数
- 组归一化的计算公式：

$$E_{ng}[\mathbf{X}] = \frac{1}{(C/G)HW} \sum_{c=gC/G+1}^{(g+1)C/G} \sum_{h=1}^H \sum_{w=1}^W \mathbf{X}_{\cdot,h,w,c}$$

$$Var_{ng}[\mathbf{X}] = \frac{1}{(C/G)HW} \sum_{c=gC/G+1}^{(g+1)C/G} \sum_{h=1}^H \sum_{w=1}^W (\mathbf{X}_{\cdot,h,w,c} - E_{ng}[\mathbf{X}])^2$$

$$\mathbf{Y}_{\cdot,h,w,(gC/G+1):(g+1)C/G} = \frac{\mathbf{X}_{\cdot,h,w,(gC/G+1):(g+1)C/G} - E_{ng}[\mathbf{X}]}{\sqrt{Var_{ng}[\mathbf{X}] + \epsilon}} \\ \mathbf{Z}_{n,h,w,\cdot} = \mathbf{Y}_{n,h,w,\cdot} \times \boldsymbol{\gamma} + \boldsymbol{\beta}$$



组归一化

□ 其中, $g \in \mathbb{N}, g = 0, \dots, G - 1$ 。可以发现, 当 G 等于1时, 其等价于层归一化; 当 $G = C$ 时, 其等价于实例归一化。



权重归一化

- 权重归一化（Weight Normalization）将权值 w 分为一个方向分量 v 和一个范数分量 g ，使用优化器分别优化这两个参数，如下所示：

$$w = \frac{g}{\|v\|} v$$

- 其中 v 是与 w 同维度的向量， $\|v\|$ 是欧式范数，因此 v 决定了 w 的方向。
- g 是标量，决定了 w 的长度。
- 由于 $\|w\|=|g|$ ，因此这一权重分解的方式将权重向量的欧氏范数进行了固定，从而实现了正则化的效果。



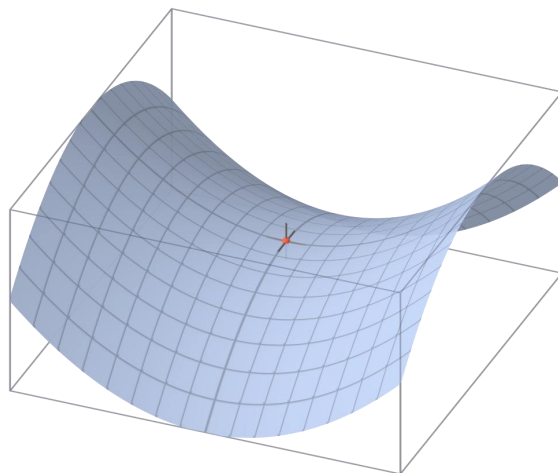
权重归一化

- ❑ 权重归一化的操作作用在了权值矩阵上，它可以带来更快的收敛速度，更强的学习率鲁棒性，能应用在循环神经网络等动态网络中。
- ❑ 并且它与样本量无关，所以可以使用较小的批大小。
- ❑ 但是权重归一化并没有对得到的特征范围进行约束的功能，所以对参数的初始值非常敏感。



深度学习中的优化算法

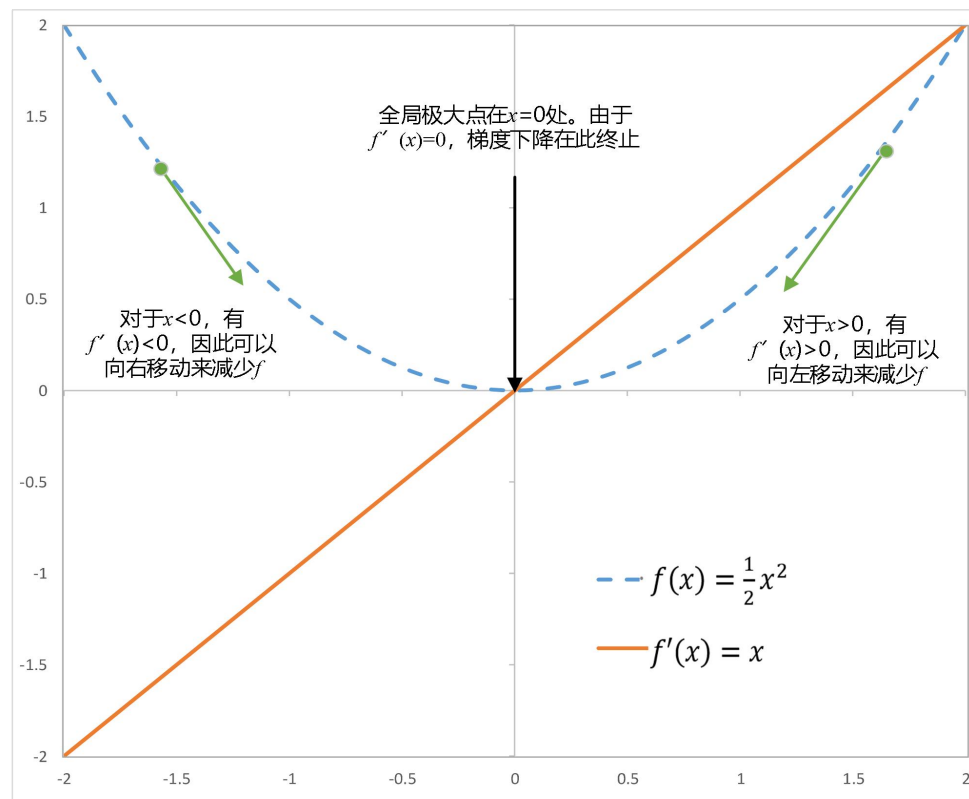
- ❑ 深度学习中的优化算法是指在模型训练过程中，通过调整参数以最小化损失函数来提高模型性能的算法。
- ❑ 深度学习中的优化算法选择直接影响了模型的收敛速度和性能。一个好的优化算法应该能够控制梯度下降的方向，避免局部最小值和鞍点，并能够加快学习的速度。



鞍点

梯度下降法

- 以 $f(x) = \frac{1}{2}x^2$ 为例展示了梯度下降法中梯度下降的实际情况，图中的箭头表示梯度下降的方向。
- 可以将 x 往导数的反方向移动一小步来减小 $f(x)$ ，即取 $x' = x - \epsilon f'(x)$ 为新的参考点，这种技术称为梯度下降。



梯度下降法

□ 梯度下降的步骤如下：

- 1.初始化参数值。
- 2.计算当前参数下损失函数的梯度。
- 3.按照梯度的方向和学习率对参数进行更新，学习率决定了每一步参数调整的大小。
- 4.重复步骤2和3，直到达到预定的停止条件（如达到最大迭代次数或损失函数收敛到指定的值）。
- 5.通过不断更新参数，梯度下降法能够使模型逐渐朝着更小的损失函数值移动，从而找到损失函数的局部最小值或全局最小值。



梯度下降法

- 需要注意的是，梯度下降法可能会陷入局部最小值而无法到达全局最小值。为了解决这个问题，可以使用不同的改进方法，
- 批量梯度下降法（Batch Gradient Descent, BGD）
 - 随机梯度下降法（Stochastic Gradient Descent, SGD）
 - 小批量随机梯度下降法（Mini-batch Stochastic Gradient Descent, mini-batch SGD）



批量梯度下降法

- 批量梯度下降法（Batch Gradient Descent, BGD）是在更新参数的时候使用所有的样本进行更新，假设一个批次中有n个样本，则计算梯度的时候使用这个批次中所有n个样本的梯度数据，计算公式：

$$x' = x - \epsilon \sum_{j=1}^n \nabla_{x_j} f(x_j)$$

- 因为需要计算整个批次的梯度来执行一次更新，所以批量梯度下降法可能非常缓慢，并且对于内存不适合的数据集来说是难以处理的，而且批量梯度下降法也不允许在线更新模型。



随机梯度下降法

- 随机梯度下降法（Stochastic Gradient Descent, SGD）与批量梯度下降法相对应，区别在于求梯度时没有用一个批次中的所有n个样本数据，而是随机选取其中一个样本来求梯度，计算公式：

$$x' = x - \epsilon \nabla_{x_j} f(x_j)$$

- 由于每次仅采用一个样本来迭代，训练速度很快。但是基于这种方法的模型对每个实例都非常敏感，造成了收敛中的不稳定，来回波动，当然实例引发的波动可能使模型跳过皱褶，找到更好的局部最小值。
- 已有研究表明，当缓慢降低学习率时，SGD表现出与批量梯度下降法相同的收敛行为，对于非凸优化和凸优化，几乎可以分别收敛到局部最小值或全局最小值。



小批量随机梯度下降法

- 小批量随机梯度下降法（Mini-batch Stochastic Gradient Descent, mini-batch SGD）是批量梯度下降法和随机梯度下降法的折衷，也就是对于一个批次中的所有n个样本，采用k个样本来迭代：

$$x' = x - \epsilon \sum_{j=t}^{t+k-1} \nabla_{x_j} f(x_j)$$

- 小批量随机梯度下降法一方面减少了参数更新的方差，使得损失函数更稳定的收敛，另一方面可以利用最先进的深度学习库中常见的矩阵优化，使梯度的计算非常高效。常见的小批量大小在50-256之间，但不同的数据集可能会有所不同。



动量法

- ❑ 随机梯度下降法容易导致模型陷入局部最优或者鞍点，且梯度更新不稳定，容易陷入震荡。
- ❑ 针对此问题，动量（Momentum）法提出在随机梯度下降法的基础上，引入动量的概念，使用指数移动平均值取代梯度计算，计算公式：

$$x' = x - \epsilon m_t = x - \epsilon \left(\beta m_{t-1} + \nabla_{x_j} f(x_j, t) \right)$$

- 其中 ϵ 表示学习率， m_t 即为引入的动量，它累加了过去的梯度信息：

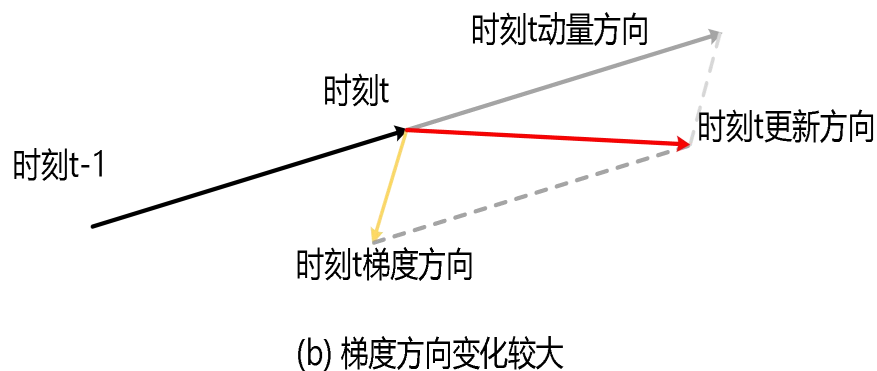
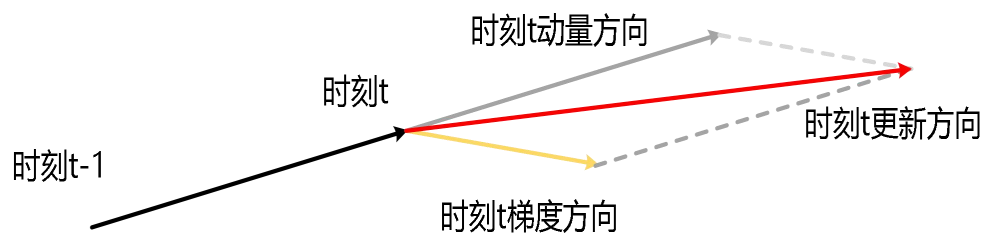
$$\begin{aligned} m_t &= \beta m_{t-1} + \nabla_{x_j} f(x_j, t) = \beta \left(\beta m_{t-2} + \nabla_{x_j} f(x_j, t-1) \right) + \nabla_{x_j} f(x_j, t) \\ &= \sum_{\tau=0}^{t-1} \beta^\tau \nabla_{x_j} f(x_j, t-\tau) \end{aligned}$$



动量法

□ 直观来看，动量在当前时刻的梯度上，增加了与先前时刻梯度相关的 βm_{t-1} ，即当前时刻参数的更新方向不仅由当前梯度方向决定，也与之之前累积的梯度方向（动量方向）相关。

□ 如图所示，对于参数中梯度方向变化不大的维度，动量的引入可以加速参数收敛；而针对梯度方向变化较大的维度，则可以缓解震荡现象。



NAG

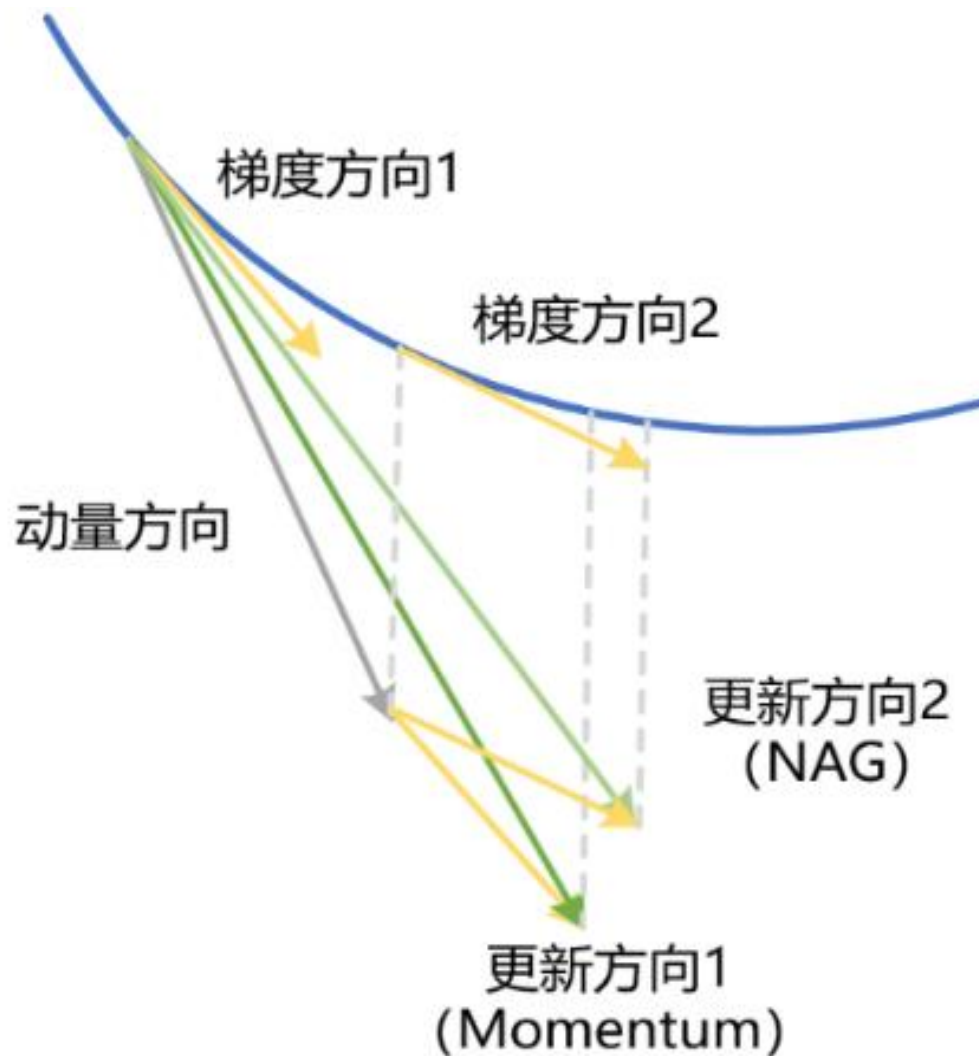
□ NAG (Nesterov Accelerated Gradient) 针对动量法进行了进一步改进，动量法中计算的为当前时刻的梯度，NAG提出计算参数经过更新之后时刻的梯度：

$$x' = x - \epsilon m_t = x - \epsilon \left(\beta m_{t-1} + \nabla_{x_j} f(x_j - \beta m_{t-1}, t) \right)$$



NAG

- ❑ 动量法与NAG参数更新方向的比较如图所示，对于动量法，参数更新方向1由当前步的动量方向和梯度方向1决定。
- ❑ 对于NAG，其相较于动量法多了一步前瞻操作，即先对当前步应用动量方向进行参数更新以得到梯度方向2，然后结合动量方向得到参数更新方向2。



AdaGrad

- ❑ 以上方法均使用相同的学习率对所有参数进行更新，但是对于不常见的参数，由于其更新频率低可能导致其无法更新到较好的状态。
- ❑ 解决此问题的一个思路是在训练过程中对不同参数的学习率进行动态调整：对于常见参数，由于其更新频率高从而调低其学习率；对于不常见参数，由于其更新频率低从而调高其学习率。
- ❑ 可以记录截至第 t 步参数 x 被更新的次数 $s(\Theta, t)$ ，并将其学习率设置为 $\epsilon = \frac{\epsilon_0}{\sqrt{s(\Theta, t) + c}}$ ，其中， ϵ_0 为设置的初始全局学习率， c 为一个较小的正数，常设置为 10^{-6} ，用于保证数值稳定性，避免分母为零。



AdaGrad

- AdaGrad (Adaptive Gradient) 算法提出使用梯度的平方和作为 $s(\Theta, t)$ 的替换估计, 记为 s_t :

$$x' = x - \frac{\epsilon_0}{\sqrt{s_t + c}} \nabla_{x_j} f(x_j, t)$$

$$s_t = s_{t-1} + \nabla_{x_j} f(x_j, t)^2$$

- AdaGrad也存在一定问题: 首先AdaGrad依赖于初始全局学习率 ϵ_0 , 如果初始全局学习率太大, 会使得训练初期对梯度的调节太大; 此外随着训练次数的迭代, 累积的梯度平方和会越来越大, 会使得学习率单调递减至0, 从而导致训练提前结束。



RMSProp

- 为解决AdaGrad中存在的梯度累计导致的学习率递减至0的问题，RMSProp（Root Mean Square Prop）改用指数移动平均改善学习率动态调整过程：

$$x' = x - \frac{\epsilon_0}{\sqrt{s_t + c}} \nabla_{x_j} f(x_j, t)$$

$$s_t = \gamma s_{t-1} + (1 - \gamma) \nabla_{x_j} f(x_j, t)^2$$

- 其中， γ 为指数移动平均的加权系数，且 $\gamma \in [0, 1]$ 。
RMSProp改善了AdaGrad存在的问题，但是仍然需要手动指定初始学习率 ϵ_0 。



ADADELTA

□ ADADELTA是AdaGrad的另一种变体，同时使用了两个状态变量， s_t 与RMSProp一样用于存储梯度平方和（二阶动量）的指数移动平均值， Δx_t 用于存储模型本身参数更新变量平方和的指数移动平均值。

$$x' = x - \frac{\sqrt{\Delta x_{t-1} + c}}{\sqrt{s_t + c}} \nabla_{x_j} f(x_j, t)$$

$$s_t = \gamma s_{t-1} + (1 - \gamma) \nabla_{x_j} f(x_j, t)^2$$

$$\Delta x_{t-1} = \gamma \Delta x_{t-2} + (1 - \gamma) g_{t-1}^2$$

— 其中

$$g_{t-1} = \frac{\sqrt{\Delta x_{t-2} + c}}{\sqrt{s_{t-1} + c}} \nabla_{x_j} f(x_j, t - 1)$$

□ 从上式可看出，ADADELTA无需人工指定学习率。



ADAM

- 动量法在SGD的基础上引入了一阶动量，AdaGrad、RMSProp和ADADELTA则引入了二阶动量。ADAM (Adaptive Moment) 则同时利用了梯度的一阶动量和二阶动量，其参数更新公式如下：

$$x' = x - \epsilon \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + c}$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{x_j} f(x_j, t), \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) \nabla_{x_j} f(x_j, t)^2, \quad \hat{s}_t = \frac{s_t}{1 - \beta_2^t}$$

- 其中， β_1 和 β_2 为非负超参数，默认设置 $\beta_1=0.9$ 、 $\beta_2=0.999$ ， m_t 和 s_t 为参数梯度的一阶动量和二阶动量的指数移动平均值，即对参数梯度的一阶动量期望和二阶动量期望的估计。



ADAM

- 但是由于初始值 m_0 和 s_0 设置为0，当 t 较小时存在较大的初始偏差，从而估计值并不准确。例如，当 $\beta_1=0.9$ 时， $m_1=0.1\nabla_{x_j}f(x_j, 1)$ 。为消除这样的影响，研究人员提出使用 m_t 和 s_t 的偏差修正值 $\hat{m}_t$ 和 $\hat{s}_t$ 对参数进行更新。



NADAM

- NADAM (Nesterov ADAM) 在ADAM的基础上引入了NAG的思想，引入“未来时刻”的梯度，首先将ADAM中的 $\hat{m}_t$ 展开，参数更新公式：

$$x' = x - \epsilon \frac{1}{\sqrt{\hat{s}_t} + c} \left(\frac{\beta_1 m_{t-1}}{1 - \beta_1^t} + \frac{(1 - \beta_1) \nabla_{x_j} f(x_j, t)}{1 - \beta_1^t} \right)$$

- NADAM则是将其中的一阶动量 m_{t-1} 替换为 m_t （前瞻计算）：

$$x' = x - \epsilon \frac{1}{\sqrt{\hat{s}_t} + c} \left(\frac{\beta_1 m_t}{1 - \beta_1^t} + \frac{(1 - \beta_1) \nabla_{x_j} f(x_j, t)}{1 - \beta_1^t} \right)$$

$$x - \epsilon \frac{1}{\sqrt{\hat{s}_t} + c} \left(\beta_1 \hat{m}_t + \frac{(1 - \beta_1) \nabla_{x_j} f(x_j, t)}{1 - \beta_1^t} \right)$$





8

中英文术语对照



中英文术语对照

- ☐ 正则化: **Regularization**
- ☐ 优化: **Optimization**
- ☐ 过拟合: **Over-fitting**
- ☐ 欠拟合: **Under-fitting**
- ☐ 合适拟合: **Appropriate-fitting**
- ☐ 偏差: **Bias**
- ☐ 方差: **Variance**
- ☐ 泛化误差: **Generalization Error**
- ☐ 噪声: **Noise**



中英文术语对照

- ☐ 轮: Epoch
- ☐ Lasso回归: Lasso Regression
- ☐ 岭回归: Ridge Regression
- ☐ 数据增强: Data Augmentation
- ☐ 自助采样法: Bootstrap Sampling
- ☐ 提前终止: Early Stopping
- ☐ 归一化: Normalization
- ☐ 最大最小归一化: Max-min Normalization



中英文术语对照

- ❑ 标准归一化: **Standard Normalization**
- ❑ 批归一化: **Batch Normalization, BN**
- ❑ 层归一化: **Layer Normalization, LN**
- ❑ 实例归一化: **Instance Normalization**
- ❑ 组归一化: **Group Normalization**
- ❑ 权重归一化: **Weight Normalization**
- ❑ 梯度下降法: **Gradient Descent, GD**
- ❑ 批量梯度下降法: **Batch Gradient Descent, BGD**
- ❑ 随机梯度下降法: **Stochastic Gradient Descent, SGD**
- ❑ 小批量梯度下降法: **Mini-batch Stochastic Gradient Descent, mini-batch SGD**
- ❑ 动量: **Momentum**



谢谢！



计算机科学与技术学院

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY